Vision, Roadmap & Release Plan

Project Name:

KnowledgeVault

Document Control

Change Record

Date	Author	Version	Change Reference
09/21/2026	Durvank Deorukhkar, Percival Tapera	1.0	Initial draft of Vision, Roadmap & Release Plan

Reviewers

Sign Off Date	Reviewer	Position	Sign Off
	Sufyan Almajali	Course Instructor / Project Advisor	
	Dikshya Shrestha	Teaching Assistant	

Distribution

Copy Number	Name
1	Durvank Deorukhkar, Team Member
2	Percival Tapera, Team Member
3	Sufyan Almajali, Instructor / Advisor

Table of Contents

DOCUMENT CONTROL ..... **II**

Change Record ii

Reviewers ii

Distribution ii

TABLE OF CONTENTS ..... **III**

1 PRODUCT VISION **5**

1.1 Vision Statement 5

1.2 Target Users 5

1.3 Core Value Proposition 5

1.4 Differentiation 5

1.5 Product Type: SaaS vs. Targeted Deployment 6

2 PRODUCT ROADMAP **7**

2.1 Roadmap Themes 7

2.2 Now / Next / Later View 7

3 RELEASE PLANNING **9**

3.1 Release 1: Internal Alpha 9

3.2 Release 2: Pilot Beta 9

3.3 Release 3: Final / Capstone Submission 9

4 PROJECT MANAGEMENT TOOL SETUP (JIRA) **11**

4.1 Project and Board Structure 11

4.2 Epics (by Roadmap Theme) and Components 11

4.3 Issue Template 11

5 GITHUB REPOSITORY **13**

5.1 Repository Structure 13

5.2 Initial Sample Code 13

6 LANGUAGES AND TOOLS **14**

6.1 Languages 14

6.2 Frameworks and Libraries 14

6.3 Data and Infrastructure 14

6.4 Collaboration and Quality Tools 14

7 INITIAL TASK/ROLE LIST 15

1 Product Vision

1.1 Vision Statement

For graduate students, researchers, and knowledge workers who accumulate large personal collections of notes, PDFs, and articles, KnowledgeVault is a personal knowledge management web application that lets users retrieve their own material by meaning rather than by exact keyword. Unlike folder-based storage and traditional keyword-search note apps, KnowledgeVault automatically tags and summarizes every uploaded document and indexes it for semantic search, so a user can find what they need using plain-language queries that reflect how they actually think about a topic, not how the file happened to be named.

1.2 Target Users

- Graduate students and researchers who collect readings, notes, and reference material across multiple courses or projects over a semester or longer.
- Knowledge workers who save articles, reports, and internal documents and later struggle to relocate a specific one from memory alone.
- Anyone maintaining a personal archive who currently relies on manual folder structures or exact-match search and finds that approach breaking down as the archive grows.

1.3 Core Value Proposition

- Meaning-based retrieval: a plain-language query returns relevant material even when it shares no exact words with the source document.
- Zero-effort organization: tags and summaries are generated automatically at upload time, with no manual filing required.
- A single trusted archive: one searchable place for notes, PDFs, and articles, replacing scattered folders and apps.

1.4 Differentiation

Consumer note apps and cloud drives support only exact keyword search, and commercial personal-knowledge tools that add smarter search generally do so as closed, non-extensible products. KnowledgeVault is purpose-built around semantic search as its core feature rather than an add-on, using a vector database and embeddings to match on meaning, while remaining a lightweight, single-user tool rather than an enterprise platform.

1.5 Product Type: SaaS vs. Targeted Deployment

KnowledgeVault is being architected with SaaS principles from the start (account-based authentication and per-user data isolation, see Section 2.1, Theme 4) rather than as a single-tenant tool built for one company. For the capstone, the team is not building or selling a commercial product for a specific paying client; the deliverable is a working multi-user web application, deployed as one hosted instance, aimed at a general user base of graduate students, researchers, and knowledge workers. This keeps the architecture SaaS-ready for a possible future product while keeping the semester scope realistic for a two-person team.

2 Product Roadmap

The roadmap is organized into four themes, each corresponding to one two-week sprint defined in the Project Charter's timeline, running from the charter approval in mid-September through final submission in December.

2.1 Roadmap Themes

Theme	Timeframe	Goal	Key Features
Theme 1: Foundation and Ingestion	Sprint 1 09/24 - 10/01	Stand up the data layer, get documents into the system, and scaffold the frontend application in parallel rather than after the backend.	Upload endpoint for notes, PDFs, and articles Text extraction and chunking pipeline Initial data model (documents, chunks, metadata) React application scaffold with a basic upload screen wired to the endpoint
Theme 2: Semantic Search and Browse UI	Sprint 2 10/01 - 10/15	Make uploaded content findable by meaning and self-organizing, and give users a first working browse and search screen wired to live backend results.	Embedding generation for chunks Vector similarity search API Automatic tagging and summarization React browse/filter view and initial search UI, connected to the live search API
Theme 3: User Experience Refinement	Sprint 3 10/15 - 10/29	Refine and integrate the frontend started in Sprints 1-2 into a cohesive, polished capture, browse, and search experience.	Polished upload and capture flow with validation and error states Refined browse/filter view by tag and recency Result previews and end-to-end integration testing across frontend and backend
Theme 4: Deployment, Security, and Polish	Sprint 4 10/29 - 11/12	Make the system reachable, secure, and ready for evaluation.	AWS deployment (backend, frontend, vector DB) Authentication and per-user data isolation Bug fixes from pilot feedback and final polish

2.2 Now / Next / Later View

Now (Sprint 1-2)	Next (Sprint 3)	Later (Sprint 4+)
Ingestion pipeline	UI refinement and end-to-end	AWS deployment

Data model Embeddings and vector search Auto-tagging/summarization React app scaffold, upload and browse UI	integration Search results UI polish Pilot testing with volunteers	Auth and data isolation Accessibility and security hardening Post-capstone: third-party note-app integrations (out of scope for this course)

Roadmap items are re-prioritized at the start of each sprint based on the prior sprint's status report and any advisor feedback from the biweekly instructor meeting, per the syllabus schedule.

3 Release Planning

Development is organized into three releases that build on one another, aligned to the sprint boundaries in Section 2.1. Each release has a stated goal, the backlog items pulled into it, and entry/exit criteria.

3.1 Release 1: Internal Alpha

Attribute	Detail
Target date	End of Sprint 2 (10/15/26)
Goal	Prove the core technical bet: a document can be uploaded, embedded, and retrieved by a plain-language query.
Scope	Upload and text extraction pipeline Embedding generation and vector search API Automatic tagging and summarization React application scaffold with a basic upload screen (not yet styled or polished)
Exit criteria	A curated test set of documents can be uploaded through the basic UI and queried in plain language to return correct matches, verified by the team.

3.2 Release 2: Pilot Beta

Attribute	Detail
Target date	End of Sprint 3 (10/29/26)
Goal	Expand the basic interface from Release 1 into a full upload, browse, and search experience, and validate it with real pilot users.
Scope	Full React upload, browse, and search interface (built on the Release 1 scaffold) End-to-end flow: sign in, upload, search, view result Pilot testing session with volunteer users
Exit criteria	At least 80% of pilot participants can upload an item and locate it later using a plain-language query without facilitator help.

3.3 Release 3: Final / Capstone Submission

Attribute	Detail
Target date	End of Sprint 4 through final submission (11/12 - 12/10/26)
Goal	Ship a deployed, secure, demo-ready version of KnowledgeVault for grading and presentation.

Scope	AWS deployment with a working URL Authentication and per-user data isolation Defect fixes from pilot feedback and final end-to-end testing Final documentation, demo video, and GitHub repository
Exit criteria	Application is reachable at a deployed AWS URL with core flows working end-to-end; all critical/high defects resolved or documented.

4 Project Management Tool Setup (Jira)

The team will use Jira to run the project according to Scrum. This section documents the project structure to be created; the live Jira project itself is a separate deliverable shared directly with the instructor and TA.

4.1 Project and Board Structure

Jira project: "KnowledgeVault Capstone, Fall 2026" (key: KV), Scrum template

Board Element	Purpose
Backlog (native Jira view)	All identified stories and tasks, organized into epics and prioritized top to bottom, not yet pulled into a sprint.
Sprint (native Jira feature)	A two-week iteration configured in Jira, matching the sprint boundaries defined in Section 2.1.
To Do	Issues committed to the active sprint but not yet started.
In Progress	Issues currently being worked on; each issue has an assignee.
In Review / Testing	Work completed and awaiting code review, integration testing, or advisor feedback.
Done	Completed and verified issues, kept per sprint for the sprint review/status report.

4.2 Epics (by Roadmap Theme) and Components

One epic per Roadmap Theme from Section 2.1: Epic 1: Foundation and Ingestion, Epic 2: Semantic Search and Browse UI, Epic 3: User Experience Refinement, Epic 4: Deployment, Security, and Polish.

- Component: Backend/ML: ingestion, embeddings, search API, tagging/summarization (Durvank)
- Component: Frontend: React UI, capture, browse, search views (Percival)
- Component: DevOps/Infra: AWS deployment, CI, environment configuration (Percival)
- Component: Docs/Testing: documentation, test cases, status reports (Both)
- Issue types used: Story (user-facing capability), Task (non-user-facing work), Bug (defect found during testing)

4.3 Issue Template

Every issue will use a consistent template so the board stays readable across both members and the advisor:

- Summary: short user story or task name
- Description: "As a [user], I want [capability], so that [benefit]"
- Acceptance criteria: a short checklist of what "done" means
- Epic link: per Section 4.2
- Assignee and story point estimate (Jira's native Story Points field)
- Sprint: aligned to the sprint end date from Section 2.1

Action item: this project will be created directly in Jira and shared with the instructor (Sufyan Almajali) and TA (Dikshya Shrestha) as project members, satisfying the requirement to set up and share an online project management solution according to the Agile method. This document defines the structure the live Jira project follows.

5 GitHub Repository

A GitHub repository will be created to host the project's codebase for the remainder of the capstone, starting with a sample scaffold rather than the full application.

5.1 Repository Structure

Path	Contents
/backend	FastAPI application: sample "health check" and placeholder upload endpoint, project dependencies (requirements.txt), and a starter folder layout for the ingestion and search services to come.
/frontend	React application scaffold (created via Vite), with a placeholder landing page and project dependencies (package.json).
/docs	Project Charter, this Vision/Roadmap/Release Plan, and future documentation deliverables.
/.github/workflows	Placeholder GitHub Actions workflow file for continuous integration, to be built out once tests exist.
README.md	Project overview, setup instructions, and links to the team's Jira project and documentation.

5.2 Initial Sample Code

For this submission, the repository contains sample/placeholder code only, consistent with the assignment's scope at this stage of the project:

- Backend: a minimal FastAPI app with a single GET /health endpoint returning a status message, demonstrating the chosen framework and project layout.
- Frontend: a minimal React app rendering a placeholder KnowledgeVault landing page, demonstrating the chosen framework and build tooling.

Full ingestion, embedding, search, and UI functionality will be built incrementally starting in Sprint 1, per the roadmap in Section 2.

Action item: the repository will be created under the team's GitHub account and its link shared with the instructor and TA, satisfying the requirement to prepare a GitHub site with sample code.

6 Languages and Tools

6.1 Languages

- Python: backend services, ingestion pipeline, and embedding/search logic
- TypeScript / JavaScript: React frontend
- SQL: data model and queries against the metadata store

6.2 Frameworks and Libraries

- FastAPI: backend REST API framework
- React (with Vite): frontend framework and build tooling
- An embeddings library/API (evaluated in Sprint 1): for generating vector representations of document chunks

6.3 Data and Infrastructure

- A vector database (open-source or managed, evaluated per the Charter's Options Analysis): semantic search index
- PostgreSQL: relational metadata storage (documents, tags, users)
- Amazon Web Services (AWS): compute, storage, and hosting for the deployed application
- Docker: containerizing the backend for consistent local development and deployment

6.4 Collaboration and Quality Tools

- GitHub: source control and issue tracking
- GitHub Actions: continuous integration once a test suite exists
- Jira: Scrum board, sprint tracking, and backlog management
- Pytest and a frontend test runner (e.g., Jest/Vitest): automated testing

7 Initial Task/Role List

Initial division of responsibility for the upcoming sprints, broken down by role rather than only by person, consistent with the staffing plan in the Project Charter. Roles marked "Both" are shared, but the tasks under each role are still tracked separately in Jira so ownership stays clear even where the same two people cover multiple roles.

Role	Owner	Description	Initial Tasks (Sprint 1)
Backend API Development	Durvank	Own the FastAPI service, upload endpoint, and text-extraction pipeline.	Set up FastAPI project scaffold Build upload endpoint and text extraction Define initial data model
ML / Semantic Search	Durvank	Own embeddings, vector search, and automatic tagging/summarization.	Finalize vector database choice Prototype embedding generation on sample documents
Backend Cloud Infrastructure	Durvank	Own AWS backend deployment, environment config, and per-user data isolation.	Draft AWS backend deployment plan Set up dev/staging environment variables
Frontend Development	Percival	Own the React application: capture, browse, and search screens.	Set up React project scaffold Draft initial wireframes for capture/browse/search
Frontend Cloud Infrastructure / DevOps	Percival	Own AWS frontend hosting and CI/CD pipeline configuration.	Configure GitHub repository and CI placeholder Draft CI pipeline outline for later sprints
QA / Testing Coordination	Percival	Own test-plan coordination and pilot-testing logistics in later sprints.	Draft initial manual test-case checklist
Project and Sprint Management	Both	Shared ownership of sprint planning, Jira upkeep, and status reporting.	Create and share the Jira project Prepare Sprint 1 kickoff and task breakdown
Technical Architecture Decisions	Both	Shared ownership of key technical choices that affect both sides of the stack.	Finalize vector database choice Agree on API contract between frontend and backend